

Home Laptop Lifetime Runbook

Oct 1, 2026 · @mate

Overview

The home laptop is a 10+ year-old Arch machine (kernel 6.17) that now works as an always-on server. It runs your task sync server, the lecture and study sites on the tailnet, and Docker, Quarto and Jupyter work, and it is the target of the work laptop's nightly restic backup. Its weak points are the ones age hits first: the disk, the battery, heat and fans, and an unattended rolling-release OS.

Do the steps in order. Step 1 (backups) comes before anything else because every later step touches a machine that could fail at any time.

1. **Baseline:** record today's health so you can see change later.
2. **Backups:** get its data and the restic repo off this one disk.
3. **Disk:** let `smartd` watch the drive and run self-tests.
4. **Battery and power:** cap the charge, never suspend, ignore the lid.
5. **Heat:** check temperatures, clean the fan, limit thermal stress.
6. **Headless services:** no desktop at boot, user services that survive logout, Emacs as a daemon.
7. **Safe updates:** a fallback kernel and a calm update routine.
8. **Work from the work laptop:** SSH, Docker context, `tailscale serve`, mosh.
9. **Monitoring:** a daily health check that warns you before something breaks.

Every command below runs on the home laptop over SSH unless marked (**work laptop**). Commands with `sudo` change the system; the rest only read or touch your user.

Step 0 — Baseline

Spend 15 minutes recording today's numbers once; every later check compares against them. Install the tools, run the commands, and save the output to

```
~/health/baseline-$(date +%F).txt.
```

1. Bring the whole system up to date FIRST (never install on top of an out-of-date system)

```
sudo pacman -Sy archlinux-keyring && sudo pacman -Su    # keyring first after
a long gap
yay -Sua                                                # rebuild/upgrade AUR
packages
sudo reboot                                            # new kernel / glibc /
gcc runtime
```

2. Then install the tools

```
sudo pacman -S --needed smartmontools lm_sensors nvme-cli upower acpi stress-
ng
sudo sensors-detect --auto
mkdir -p ~/health
```

What	Command	What to write down
Disk model, type, age	<code>lsblk -d -o NAME,MODEL,ROTA,SIZE</code> then <code>sudo smartctl -i /dev/sda</code> (or <code>/dev/nvme0n1</code>)	HDD (<code>ROTA=1</code>) or SSD, model, size
Disk health	<code>sudo smartctl -H -A /dev/sda</code>	Overall result; for an HDD: <code>Reallocated_Sector_Ct</code> , <code>Current_Pending_Sector</code> , <code>Offline_Uncorrectable</code> , <code>Power_On_Hours</code> ; for an SSD: <code>Wear_Leveling_Count</code> or <code>Percentage Used</code> , <code>Available Spare</code>
Battery wear	<code>upower -i \$(upower -e grep BAT)</code>	<code>energy-full ÷ energy-full-design</code> = health %, and <code>charge-cycles</code>
Charge-limit support	<code>ls /sys/class/power_supply/BAT*/charge_control_end_threshold</code>	Exists or not (decides Step 3)
Temperatures idle	<code>sensors</code>	CPU package and core temps, fan RPM
Temperatures under load	<code>stress-ng --cpu 0 --timeout 120s</code> in one pane, <code>watch -n2 sensors</code> in another	Peak CPU temp; above ~90 °C means Step 4 is urgent
Memory errors	<code>journalctl -k -b grep -iE 'mce edac memory error'</code>	Any hits

What	Command	What to write down
Disk and I/O errors in logs	<code>journalctl -p err -b grep -iE 'ata nvme I/O error ext4 btrfs'</code>	Any hits
Filesystem free space	<code>df -h /</code> and <code>df -h</code> on the backup drive	Free GB per filesystem
Boot time	<code>systemd-analyze</code>	Total seconds (a sudden jump hints at a failing disk)

If the disk already shows reallocated or pending sectors, or the SSD reports under 10% spare, skip ahead and finish Step 1 today.

Step 1 — Backups first

Today there is exactly one copy of the home laptop's own data, and the work laptop's backups also live only on this machine. The goal is 3-2-1: three copies, two kinds of media, one off this machine.

What must be saved from the home laptop:

- `~/srv` (lecture and study sites), `~/sys_org`, `~/.local/share/taskchampion-sync-server` (task database)
- `~/.config` (or just push the dotfiles repo) and `~/.doom.d` until it moves into the repo
- The restic repository that receives the work laptop's backups

1. Pick an off-machine destination. Either an external USB disk you keep at home (cheap, simple) or a cloud bucket such as Backblaze B2 (survives theft or fire). Ideally both.
2. If it is a USB disk, mount it the same way every time, even with nobody logged in. Label it `BACKUP`, run `sudo mkdir /mnt/backup`, and add this line to `/etc/fstab`. It mounts on first access, and boot doesn't fail if the disk is unplugged:

```
LABEL=BACKUP /mnt/backup ext4 noatime,nofail,x-systemd.automount,x-
systemd.device-timeout=10s 0 2
```

Then `sudo systemctl daemon-reload` and `sudo chown hlima: /mnt/backup`.

3. Create a restic repo for the home laptop's own data. Save its password to `~/.config/restic/password-home` (`chmod 600`; the dotfiles repo already ignores it):

```
restic -r /mnt/backup/restic-home init
restic -r /mnt/backup/restic-work init --copy-chunker-params --from-repo
<path-of-work-repo-on-this-laptop>
```

The second repo receives copies of the work laptop's backups. Sharing chunker parameters keeps the copies deduplicated.

4. Run the first backup by hand with the kit's script (`~/.config/dotfiles-kit/server/home-backup.sh`). It backs up `~/srv` , `~/sys_org` , `~/.config` , `~/.doom.d` and the task database, prunes to 7 daily, 4 weekly and 12 monthly snapshots, then copies the work repo:

```
DEST=/mnt/backup WORK_REPO=<path-of-work-repo-on-this-laptop> \
RESTIC_FROM_PASSWORD_FILE=~/.config/restic/password sh ~/.config/dotfiles-
kit/server/home-backup.sh
```

5. Automate it: copy the script and `home-backup.service` / `.timer` into `~/.config/server/` and `~/.config/systemd/user/` , set `WORK_REPO=` in the service, then `systemctl --user enable --now home-backup.timer` . It runs nightly at 02:30, after the work laptop's 01:30 backup, and Step 8 alerts you if it fails.
6. Prove a restore works once, today: `restic -r /mnt/backup/restic-home restore latest --target /tmp/restore-test --include ~/srv` and open a few files.

Once a month run `restic check --read-data-subset=5%` on each repo; it reads back a sample of the stored data to catch silent corruption.

Step 2 — Disk health

The disk is the part most likely to fail, and SMART usually warns weeks ahead. `smartd` watches it continuously and runs self-tests on a schedule.

1. Configure `smartd` . Replace the `DEVICESCAN` line in `/etc/smartd.conf` with one line for your disk:

```
/dev/sda -a -o on -S on -n standby,q -s (S/../../03|L/../../7/04) -W
4,45,55 -m root -M exec /usr/local/bin/smartd-notify
```

This runs a short test every night at 03:00 and a long test on Sundays at 04:00. It warns when the temperature jumps 4 °C, passes 45 °C, or hits 55 °C (critical). `smartd-notify` is in the kit and sends the alert to your phone (Step 8).

2. Enable it: `sudo systemctl enable --now smartd`. Check it saw the disk: `journalctl -u smartd -b`.
3. SSD only: make sure TRIM runs weekly with `sudo systemctl enable --now fstrim.timer`.
4. Read test results whenever you like: `sudo smartctl -l selftest /dev/sda`.

Act on these signs:

Sign	Meaning	Action
Reallocated_Sector_Ct or Current_Pending_Sector above 0 and rising	HDD surface is failing	Verify backups now; plan a replacement within weeks
A self-test reports <code>Completed: read failure</code>	Unreadable sectors	Replace the disk
SSD Percentage Used above 90%, or Available Spare below threshold	Flash is worn out	Replace the disk
<code>I/O error</code> or <code>ata ... failed</code> command in <code>journalctl -k</code>	Disk or cable problem	Run a long test; reseal the cable if it is a desktop-style SATA cable

Biggest single upgrade: if it still has a spinning HDD, a SATA SSD costs about 30–40 €. It makes the machine faster, cooler and less likely to die. Clone the disk to it with `dd` or Clonezilla, or reinstall from your dotfiles and restic.

Step 3 — Battery and power

A lithium battery kept at 100% and warm wears fastest and can swell. On a machine that never leaves the plug, hold it around 60% or take it out.

1. **Cap the charge if the firmware allows it.** If Step 0 found `charge_control_end_threshold`, make the limit permanent with a tmpfiles rule in `/etc/tmpfiles.d/battery-limit.conf`:

```
w /sys/class/power_supply/BAT0/charge_control_start_threshold - - - - 50
w /sys/class/power_supply/BAT0/charge_control_end_threshold   - - - - 60
```

Apply now with `sudo systemd-tmpfiles --create`, and check with `cat /sys/class/power_supply/BAT0/charge_control_end_threshold`. Use your battery's name (`BAT0` or `BAT1`). Drop the start line if that file does not exist.

2. No threshold support? Choose one:

- Keep the battery as a built-in UPS: it protects the disk from power cuts. Check it monthly for swelling.
- Remove it and run on AC only. That is safest for an old cell, but a power cut becomes a hard crash.

3. Swelling is a stop sign. A lifting touchpad, a bulging bottom case or a gap in the chassis means remove the battery today and recycle it.

4. Never suspend, ignore the lid. Create `/etc/systemd/logind.conf.d/server.conf`:

```
[Login]
HandleLidSwitch=ignore
HandleLidSwitchExternalPower=ignore
HandleLidSwitchDocked=ignore
IdleAction=ignore
```

Then `sudo systemctl restart systemd-logind` (it can end a graphical session, so do it from SSH) and `sudo systemctl mask sleep.target suspend.target hibernate.target hybrid-sleep.target`.

5. Turn the screen off. With the lid open, add `consoleblank=60` to the kernel command line in your bootloader entry so the backlight goes off after a minute.

6. Gentle power settings on AC. `sudo pacman -S tlp && sudo systemctl enable --now tlp`. Its defaults are sensible for a plugged-in laptop. Skip `powertop --auto-tune`: it can put the network adapter to sleep and drop your SSH session.

Track battery health monthly from `upower : energy-full` falling quickly, or the battery dying in minutes, means it no longer works as a UPS.

Step 4 — Heat and fans

Heat is what ages every other part: the disk, the battery, capacitors and the fan bearing. A ten-year-old laptop almost certainly has a dusty heatsink and dried thermal paste.

1. Placement. Put it on a hard surface with space under and behind it, or on a laptop stand, never on fabric. Most laptops pull air from the bottom.

2. **Clean it, once a year.** Blow out the fan and heatsink with compressed air, holding the fan blades still. If you're comfortable opening it, replacing the thermal paste on a 10-year-old CPU often drops load temperatures by 10–15 °C. Repeat the Step 0 load test afterwards to see the gain.
3. **Keep Intel CPUs out of the red.** On Intel, `sudo pacman -S thermald && sudo systemctl enable --now thermald` throttles gently before the hardware panics.
4. **Cap heavy jobs.** Docker builds, Quarto renders and long Python runs are what heat it up. Run them at lower priority:

```
nice -n 10 quarto render
systemd-run --user --scope -p CPUQuota=200% python long_job.py    # max 2
cores
```

In `docker compose`, add `cpus: "2"` to services that don't need the whole machine.

5. **Watch the fan.** A fan that grinds, rattles, or shows 0 RPM in `sensors` under load needs replacing. They cost 10–20 € for most models; search the laptop model plus "cooling fan".

Targets: under 50 °C idle and under 85 °C under sustained load. Step 8's daily check warns above 85 °C.

Step 5 — Headless services

Run only what a server needs, and make your own services start at boot without anyone logging in.

1. **Boot to the console, not bspwm.** Run `sudo systemctl set-default multi-user.target`. If a display manager is enabled (`systemctl status display-manager`), disable it. You can still run `startx` when you sit at it.
2. **Let your user services run without a login.** Run `sudo loginctl enable-linger hlima` and check that `loginctl show-user hlima -p Linger` prints `Linger=yes`. Without it, `tasksync`, `lectures-web` and `study-web` stop when your last SSH session closes and don't start after a reboot.
3. **Check them after every reboot:**

```
systemctl --user list-units --type=service --state=running
systemctl --user list-timers
systemctl --failed; systemctl --user --failed
```

4. **Fewer disk writes.** Cap the journal in `/etc/systemd/journald.conf.d/size.conf` with `[Journal]` and `SystemMaxUse=500M`, then `sudo systemctl restart systemd-journald`.

5. **Emacs as a daemon.** Arch's emacs package ships a user unit:

```
doom env                                # snapshot your shell PATH for
the daemon
systemctl --user enable --now emacs
echo "alias e='emacsclient -t -a \"\"'" >> ~/.bashrc  # or your zsh rc /
~/.config/shell
```

`e file.qmd` then opens instantly in the terminal, and buffers and REPLs survive when SSH drops. Reload Doom after config changes with `systemctl --user restart emacs`.

6. **tmux keeps your work alive.** Start every SSH session with `tmux new -A -s main`. Add `tmux-resurrect` and `tmux-continuum` so sessions also come back after a reboot.

Step 6 — Safe Arch updates

A rolling release on an unattended box is safe with three habits: a fallback kernel, small regular updates, and never updating right before you need the machine.

1. **Fallback kernel.** Run `sudo pacman -S linux-lts linux-lts-headers`. With GRUB, regenerate the menu (`sudo grub-mkconfig -o /boot/grub/grub.cfg`). With `systemd-boot`, copy your entry to `arch-lts.conf` pointing at `vmlinuz-linux-lts` and `initramfs-linux-lts.img`. Test-boot it once.
2. **Keep the package cache small.** Run `sudo pacman -S pacman-contrib && sudo systemctl enable --now paccache.timer`, which keeps the last 3 versions of each package.
3. **Weekly update routine**, with time to fix things afterwards:
 - a. Read the [Arch news](#) for manual steps.
 - b. `sudo pacman -Syu`, then `yay -Sua` for AUR packages. Never `pacman -Sy <pkg>` alone: partial upgrades break things.
 - c. `pacdiff` to merge any `.pacnew` config files.
 - d. If the kernel, systemd or the NVIDIA/firmware packages changed, reboot and confirm that SSH, Tailscale and your user services came back.

4. **Snapshots, if the root filesystem is btrfs** (`findmnt -no FSTYPE /`): install `snapper` and `snap-pac` so every pacman run takes a before/after snapshot you can roll back to. On ext4, your restic backups are the safety net.
5. **Keep a rescue stick.** Put a current Arch ISO on a USB stick next to the laptop. With the bad keyboard, plan to plug in a USB keyboard and an external monitor for rescue work.

Step 7 — Working from the work laptop

Let the home laptop do the heavy lifting while the work laptop only shows the results. All of this goes over Tailscale, so nothing is exposed to the internet.

1. **(work laptop) One SSH entry with connection reuse**, in `~/.ssh/config`:

```
Host home
  HostName home          # your Tailscale machine name
  User hlima
  ControlMaster auto
  ControlPath ~/.ssh/cm-%r@%h:%p
  ControlPersist 10m
  ServerAliveInterval 30
  LocalForward 8888 localhost:8888    # Jupyter
  LocalForward 4200 localhost:4200    # quarto preview
```

After the first `ssh home`, every new `ssh`, `scp`, `sshfs` and Docker call reuses the connection instantly.

2. **(work laptop) Docker runs on home, the CLI stays local:**

```
docker context create home --docker "host=ssh://home"
docker context use home      # `docker context use default` to switch back
docker ps                    # these are the home laptop's containers
```

Builds, images and volumes all live on the home laptop. Bind mounts (`-v ./data:/data`) refer to paths on the home laptop, not the work laptop.

3. **Jupyter as a service on home.** Add `~/.config/systemd/user/jupyter.service` with `ExecStart=%h/py_venv/bin/jupyter lab --no-browser --ip 127.0.0.1 --port 8888`, then `systemctl --user enable --now jupyter`. Open `http://localhost:8888` on the work laptop through the forward above. Or skip SSH and publish it on the tailnet with `tailscale serve --bg 8888`.
4. **Quarto preview.** On home, run `quarto preview doc.qmd --no-browser --port 4200` inside tmux, and open `http://localhost:4200` on the work laptop.

5. **(work laptop) Shell that survives Wi-Fi changes.** Install `mosh` on both machines and run `mosh home -- tmux new -A -s main.`
6. **(work laptop) Editing remotely.** Either `ssh home + e file` (the Emacs daemon from Step 5), or open `/ssh:home:~/project/file.qmd` in your local Doom with TRAMP. With TRAMP, Python and R REPLs started from that buffer run on the home laptop.

Step 8 — Monitoring and alerts

A daily script checks everything from Steps 0–7, logs one line per day, and sends a push notification to your phone only when something is wrong. The files are in

`~/.config/dotfiles-kit/server/`; once the dotfiles repo exists, keep them in `~/.config/server/` (already allowlisted).

File	What it does
<code>health-check.sh + .service + .timer</code>	Daily at 09:00, as root: SMART status and sectors, SSD wear, disk and CPU temperature, kernel I/O errors, battery health, free space, failed system and user units, linger, and the age of both backups. Appends a row to <code>~/health/health.csv</code> .
<code>health.env</code>	Your settings: disk device, ntfy topic, work repo path, thresholds
<code>smartd-notify</code>	Called by <code>smartd</code> (Step 2) for an immediate alert when the disk degrades
<code>notify-failure@.service</code>	Add <code>OnFailure=notify-failure@%n.service</code> to any unit to get an alert when it fails
<code>home-backup.sh + .service + .timer</code>	Step 1's nightly backup as a user unit, alerting on failure

1. **Phone alerts with ntfy.** Install the ntfy app (Android or iOS) and subscribe to a long random topic, e.g. `hlima-home-3f9a1c7e2b`. Anyone who knows the topic can read it, so keep it unguessable, or later self-host ntfy on this laptop behind Tailscale. Test it: `curl -d "hello from home" https://ntfy.sh/<your-topic>.`

2. **Install the check:**

```
cd ~/.config/dotfiles-kit/server
sudo install -m 755 health-check.sh /usr/local/bin/health-check
sudo install -m 755 smartd-notify /usr/local/bin/smartd-notify
sudo install -m 600 health.env /etc/health.env && sudo -e /etc/health.env
# set DISK, NTFY_URL, WORK_REPO
sudo install -m 644 health-check.service health-check.timer notify-
failure@.service /etc/systemd/system/
sudo systemctl daemon-reload && sudo systemctl enable --now health-
check.timer
sudo health-check          # first run: prints "All good" or the list of
warnings
```

3. Alerts for your own services. Copy `notify-failure@.service` to

`~/.config/systemd/user/` too, and put `NTFY_URL=...` in `~/.config/health/ntfy.env`. Then add `OnFailure=notify-failure@%n.service` under `[Unit]` in `tasksync`, `lectures-web`, `study-web`, `home-refresh` and `task-keep`, and run `systemctl --user daemon-reload`.

4. Watch trends, not just alarms. `column -s, -t < ~/health/health.csv | tail -30` shows the last month. Rising reallocated sectors, power-on hours climbing past ~40,000 on an HDD, falling battery health, or rising temperatures tell you to plan a replacement before anything fails.

Default thresholds: CPU $\geq 85^\circ\text{C}$, disk $\geq 50^\circ\text{C}$, free space $< 15\%$, battery health $< 40\%$, any backup older than 36 h, any reallocated or pending sector.

Routine

Once Steps 0–8 are done, upkeep is about 10 minutes a week, because the daily check and the phone alerts do the watching.

Weekly

- ☐ Arch update routine (Step 6), then confirm services came back: `systemctl --failed; systemctl --user --failed`
- ☐ Glance at the last 7 rows of `~/health/health.csv`

Monthly

- ☐ `restic check --read-data-subset=5%` on `restic-home`, `restic-work` and the work laptop's repo
- ☐ Restore one random file from the latest snapshot

- ☐ Battery: check `upower` health and look for swelling
- ☐ `sudo smartctl -l selftest /dev/sda` : the last long test passed
- ☐ Commit and push `~/.config` on both laptops

Yearly

- ☐ Blow the dust out of the fan and heatsink; repeat the Step 0 load test and compare
- ☐ Re-run the full Step 0 baseline and save it next to the old one
- ☐ Test-boot the `linux-lts` entry and the rescue USB stick
- ☐ Rotate the off-site copy (swap USB disks, or check the cloud bill and retention)
- ☐ Decide: keep this laptop another year, or move to a replacement (Plan B)

Doom config: home vs work

The home config (`~/.doom.d`) was written for an older Doom, and two of its org settings point somewhere other than the work config's. Merge them into one `~/.config/doom`, move it into the dotfiles repo, and delete `~/.doom.d` so Doom can't load the wrong one.

Area	Home (<code>~/.doom.d</code>)	Work (<code>~/.config/doom</code>)	Use in the merged config
Org directory	<code>(setq org-org "~/sys_org/")</code> : a typo, so <code>org-directory</code> stays at the default <code>~/org</code>	<code>org-directory</code> <code>"~/sys_org/"</code>	Work's line
Org-roam directory	<code>~/sys_org/</code>	<code>~/sys_org/org_roam</code>	Choose one. Two roots over the same synced notes give two different roam databases
Roam dailies, agenda, <code>org-log-done</code> , node template	Set	Not set	Home's settings
Completion	<code>company</code> + <code>ivy</code>	<code>corfu</code> + <code>orderless</code> + <code>vertico</code>	Work's (current Doom defaults)

Area	Home (~/.doom.d)	Work (~/.config/doom)	Use in the merged config
Old-Doom leftovers	(:if IS-MAC macos), the native-comp-deferred-compilation-deny-list line	Current syntax	Work's; run doom doctor to catch the rest
R / polymode	(use-package poly-R), with poly-R, ox-pandoc, ivy-bibtex and others installed through package.el (custom.el)	Doom's ess module + quarto-mode + the chunk-eval fix	Work's. Add ox-pandoc via (org +roam2 +pandoc). Declare any other package in packages.el, then empty package-selected-packages from custom.el
Org bullets	org-bullets.el copied into site-lisp/	None	(org +roam2 +pretty) instead of the vendored file
Extra packages	org-ref, org-make-toc	quarto-mode, quarto-actions	Both sets
Indentation	Spaces, tab width 4, ESS uses C++ style	Not set	Home's
Magit diff colours (same palette as your nvim diff)	Set	Not set	Home's
Identity	"John Doe" placeholder	Commented out	Set your real name and email once
Modules only on one side	lua, editorconfig, doom-quit	javascript, shell, (vc-gutter +pretty)	Union of both

Area	Home (<code>~/ .doom.d</code>)	Work (<code>~/ .config/doom</code>)	Use in the merged config
Terminal and SSH use	Nothing	Nothing	Add <code>:os (tty +osc)</code> for the Emacs daemon over SSH

To merge on the home laptop once the dotfiles repo is adopted:

1. Let the repo's `doom/` (work version) land in `~/ .config/doom`.
2. Copy over the home-only pieces from the "Home's" rows above, and fix the two org paths.
3. `mv ~/ .doom.d ~/ .doom.d.old`, then `doom sync && doom doctor`, and `systemctl --user restart emacs`.
4. When everything works for a week, delete `~/ .doom.d.old`.

Plan B: when it fails

With Steps 1 and 8 done, losing this laptop costs a day of setup on new hardware, not your data. Keep these four things true so it stays that way:

- **Everything that makes it "your server" is in git or restic.** Configs and units live in the dotfiles repo; data lives in `restic-home`. If you set something up by hand (an `fstab` line, `/etc/health.env`, `logind`), copy it into `~/ .config/server/etc/` so the repo records it.
- **Write down the hardware-specific bits** in `hosts/<name>.sh` and here: disk device, battery name, Tailscale machine name, the backup disk's label.
- **Keep the package list fresh.** `pacman -Qqe > ~/ .config/pkglists/pkglist-pacman-$(uname -n).txt` before each commit; it reinstalls everything with `pacman -S --needed - < pkglist`.
- **Know the replacement.** A used business mini PC (Lenovo M720q/M920q, Dell OptiPlex Micro, HP EliteDesk Mini with an 8th-gen Intel CPU or newer) typically sells used for roughly 100–150 € (check current listings). It idles at about 10 W, has no battery to swell, and is far faster than a 10-year-old laptop.

Migration, in order:

1. Install Arch, Tailscale and git; log into Tailscale with the old machine name (remove the old node first) so `ssh home` and your `ts.net` URLs keep working.

2. Clone the dotfiles and run `dotfiles-init.sh <remote>`; reinstall packages from the `pkglist`.
3. `restic -r /mnt/backup/restic-home restore latest --target /` for `~/srv`, `~/sys_org` and the task database.
4. Re-run Steps 2–8 (most of it is copying files from `~/.config/server/`), and point the work laptop's backup at the new disk.
5. Enable the user units and linger, reboot, and confirm the first `health-check` comes back "All good".